



UMN

INSTITUTO POLITÉCNICO DA HUÍLA

TRABALHO EM GRUPO DE ALGORITMOS E ESTRUTURA DE DADOS

TEMA: ALGORITMOS E ESTRUTURAS DE ORDENAÇÃO

Docente

Júlio César



UMN

INSTITUTO POLITÉCNICO DA HUÍLA

Curso: Engenharia Informática

Regime: Regular

2º Ano | 2º Semestre

Grupo N° 1

Sala: B6

Elementos do grupo

2024199653 - Adiandra Missina Nunda José

2024193027 - Alcides Dos Santos Mulavelwa

2024105391 - Alessandro Mbuta Papelo da Silva

2024103287 - Deolinda Graça Mangango Tchiango

2024129975 - Elizandra Do Rosário Evaristo Domingos

2023114440 - Hermes Jorge da Silva

2024103850 - Leonel Henriques Kandjuvi Bandi

2024116009 - Martins Adriano Hungulo Mussinda

ÍNDICE

INTRODUÇÃO	1
1. Conceitos fundamentais.....	2
1.1. Algoritmo.....	2
1.2. Estrutura de dados.....	2
1.3. Organização da informação	2
1.4. Ordenação	3
1.5. Conceito de chave de ordenação	3
1.6. Estabilidade dos algoritmos	3
2. Classificação dos Algoritmos de Ordenação	4
2.1. Quanto à localização dos dados	4
2.2. Quanto à complexidade computacional	4
3. Principais Algoritmos de Ordenação	5
3.1. Bubble Sort	5
3.2. Selection Sort.....	6
3.3. Insertion Sort.....	7
3.4. Shell Sort	8
3.5. Merge Sort	9
3.6. Quick Sort	10
4. Metodologia Experimental.....	11
4.1. Ambiente de Testes	11
4.2. Métricas de Avaliação	11
4.3. Implementação dos Algoritmos	11
5. Aplicações Práticas dos Algoritmos de Ordenação.....	12
5.1. Aplicações em Motores de Busca.....	12
5.2. Aplicações em Sistemas Bancários	12
5.3. Aplicações no Comércio Eletrónico	12
5.4. Aplicações em Redes Sociais.....	12
CONCLUSÕES	13
REFERÊNCIAS.....	14

INTRODUÇÃO

Atualmente, os computadores são utilizados para armazenar e processar grandes quantidades de informação. Para que esses dados possam ser utilizados de forma rápida e eficiente, é necessário que estejam bem-organizados. É nesse contexto que surgem os algoritmos e as estruturas de dados, que desempenham um papel muito importante na área da Informática.

A ordenação de dados é uma das operações mais comuns realizadas pelos sistemas computacionais. Ela consiste em organizar informações segundo um determinado critério, como ordem numérica, alfabética ou cronológica. Quando os dados estão organizados, torna-se mais fácil realizar pesquisas, consultas e outras operações, melhorando o desempenho dos sistemas.

Neste trabalho serão apresentados os principais conceitos relacionados aos algoritmos e estruturas de ordenação, bem como a sua importância na organização da informação. Serão também abordados alguns dos algoritmos de ordenação mais conhecidos, destacando as suas características, vantagens, limitações e aplicações práticas.

1. Conceitos fundamentais

1.1. Algoritmo

Um algoritmo pode ser definido como um processo sistemático destinado à resolução de um problema por meio de uma sequência finita de instruções. Em computação, os algoritmos recebem dados de entrada, executam operações previamente definidas e produzem resultados como saída.

Segundo Szwarcfiter e Markenzon (2015), um algoritmo é um processo sistemático para a resolução de um problema. Durante sua execução, ele manipula dados gerados a partir de uma entrada para produzir uma determinada saída. Além disso, os autores destacam que a eficiência de um algoritmo pode ser avaliada através da análise do tempo de execução e da memória utilizada.

Os algoritmos estão presentes em praticamente todas as aplicações computacionais modernas, desde sistemas bancários e motores de busca até aplicações móveis e redes de computadores.

1.2. Estrutura de dados

As estruturas de dados correspondem às formas utilizadas para organizar, armazenar e manipular informações em um sistema computacional. Elas permitem que os dados sejam processados de maneira eficiente, facilitando operações como inserção, remoção, pesquisa e ordenação.

De acordo com Szwarcfiter e Markenzon (2015), as estruturas de dados diferem umas das outras pela disposição e manipulação dos seus elementos. A escolha da estrutura adequada influencia diretamente o desempenho dos algoritmos e a eficiência dos sistemas computacionais.

Entre as principais estruturas de dados encontram-se: **vetores, listas, pilhas, fila, árvores, tabelas Hash**. E cada uma dessas estruturas apresenta características específicas que a tornam adequada para determinados tipos de aplicações.

1.3. Organização da informação

A organização da informação consiste na disposição adequada dos dados para facilitar o seu armazenamento, acesso e processamento. Uma organização eficiente contribui para melhorar o desempenho dos sistemas computacionais e das operações realizadas sobre os dados. Segundo Cormen et al. (2009), a eficiência de muitos

algoritmos depende diretamente da forma como os dados são organizados e estruturados.

1.4. Ordenação

A ordenação é o processo de organizar dados segundo um critério definido, como ordem crescente, decrescente, alfabética ou cronológica. Sua principal função é facilitar o acesso e o processamento das informações, tornando os sistemas mais eficientes (Souza, Ricarte & Lima, 2017).

A ordenação desempenha um papel fundamental na Ciência da Computação, pois permite organizar os dados de forma eficiente, melhorando o desempenho das buscas, consultas e análises. Além disso, muitos algoritmos de pesquisa apresentam melhores resultados quando trabalham com dados previamente ordenados (UFERSA, 2020).

1.5. Conceito de chave de ordenação

A chave de ordenação corresponde ao atributo utilizado para definir a posição dos elementos durante o processo de ordenação.

Por exemplo, em um cadastro de estudantes, os registros podem ser ordenados por:

- Nome;
- Número de matrícula;
- Média final;
- Data de nascimento.

A escolha da chave depende dos objetivos da aplicação e do tipo de informação que se deseja destacar.

1.6. Estabilidade dos algoritmos

A estabilidade é uma característica dos algoritmos de ordenação que indica se elementos com a mesma chave de ordenação mantêm a sua posição relativa após a ordenação.

Um algoritmo é considerado estável quando preserva essa ordem original; caso contrário, é considerado instável. Essa propriedade é importante em aplicações que realizam múltiplas ordenações sobre o mesmo conjunto de dados. Algoritmos como

Insertion Sort, Merge Sort e Bubble Sort são estáveis, enquanto Quick Sort, Heap Sort e Selection Sort são geralmente considerados instáveis.

2. Classificação dos Algoritmos de Ordenação

Os algoritmos de ordenação podem ser classificados segundo diferentes critérios, destacando-se a localização dos dados durante o processamento e a complexidade computacional envolvida na execução.

2.1. Quanto à localização dos dados

2.1.1. Ordenação Interna

A ordenação interna ocorre quando todos os dados estão armazenados na memória principal (RAM), permitindo acesso direto durante a execução do algoritmo. Entre os principais algoritmos de ordenação interna destacam-se Bubble Sort, Selection Sort, Insertion Sort, Shell Sort, Merge Sort e Quick Sort. Segundo Loureiro (UFMG), esse tipo de ordenação é adequado para conjuntos de dados que cabem integralmente na memória principal.

2.1.2. Ordenação Externa

A ordenação externa é utilizada quando o volume de dados excede a capacidade da memória principal, exigindo o uso de dispositivos de armazenamento secundário, como discos rígidos e SSDs. Os métodos mais utilizados incluem a intercalação balanceada e a intercalação polifásica. Essa técnica é amplamente empregada em sistemas de bases de dados e no processamento de grandes volumes de informação.

2.2. Quanto à complexidade computacional

2.2.1. Algoritmos Simples ou Elementares

São algoritmos de fácil implementação, geralmente utilizados para fins didáticos ou para pequenos conjuntos de dados. Exemplos incluem Bubble Sort, Selection Sort e Insertion Sort. Normalmente apresentam complexidade $O(n^2)$, tornando-se menos eficientes para grandes volumes de dados (Cormen et al., 2009).

2.2.2. Algoritmos Eficientes ou Avançados

São algoritmos desenvolvidos para melhorar o desempenho da ordenação em conjuntos maiores de dados. Entre os principais exemplos estão Shell Sort, Merge Sort, Quick Sort e Heap Sort. Em geral, apresentam complexidade próxima de $O(n \log n)$, oferecendo melhor desempenho que os algoritmos elementares.

2.2.3. Algoritmos Lineares

Os algoritmos lineares realizam a ordenação sem depender exclusivamente de comparações entre elementos. Exemplos incluem Counting Sort, Radix Sort e Bucket Sort. Em situações específicas, podem alcançar complexidade $O(n)$, tornando-se altamente eficientes (Cormen et al., 2009).

3. Principais Algoritmos de Ordenação

3.1. Bubble Sort

O Bubble Sort é um dos algoritmos de ordenação mais simples e conhecidos. Seu funcionamento baseia-se na comparação de elementos adjacentes, realizando trocas sempre que estes se encontram em ordem incorreta. Após cada passagem pelo vetor, o maior elemento desloca-se para o final da estrutura, repetindo-se o processo até que todos os elementos estejam ordenados.

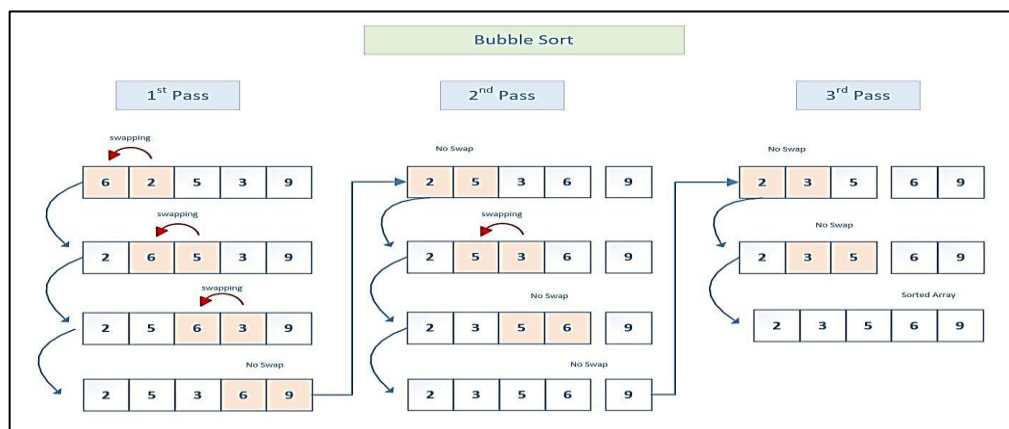


Figura 1 – Funcionamento do Bubble Sort

Vantagens

- Fácil implementação;
- Fácil compreensão;
- Adequado para fins didáticos.

Desvantagens

- Baixa eficiência para grandes volumes de dados;
- Elevado número de comparações e trocas.

Complexidade

- Melhor caso: $O(n)$
- Caso médio: $O(n^2)$

- Pior caso: $O(n^2)$

Estabilidade: Estável.

3.2. Selection Sort

O Selection Sort realiza a ordenação selecionando, a cada etapa, o menor elemento da parte não ordenada da lista e posicionando-o na sua localização correta. Esse processo é repetido sucessivamente até que todos os elementos estejam organizados.

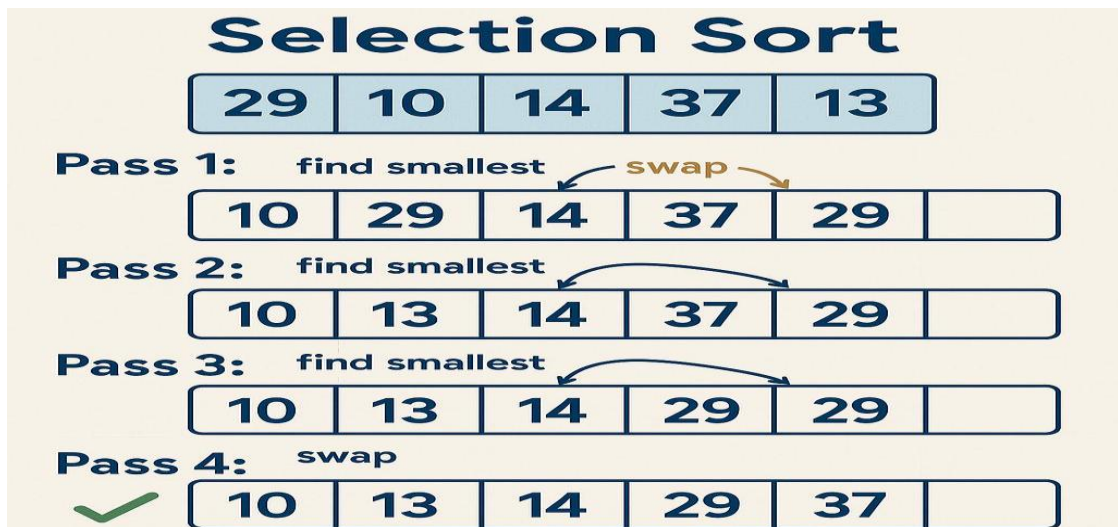


Figura 2 – Funcionamento do Selection Sort

Vantagens

- Implementação simples;
- Realiza poucas trocas de elementos;
- Baixo consumo de memória.

Desvantagens

- Grande quantidade de comparações;
- Pouco eficiente para grandes conjuntos de dados.

Complexidade

- Melhor caso: $O(n^2)$
- Caso médio: $O(n^2)$
- Pior caso: $O(n^2)$

Estabilidade: Instável.

3.3. Insertion Sort

O Insertion Sort organiza os elementos gradualmente, inserindo cada novo elemento na posição correta dentro da parte já ordenada da lista. Esse algoritmo apresenta excelente desempenho em listas pequenas ou parcialmente ordenadas (UFERSA, 2017).



Figura 3 – Funcionamento do Insertion Sort

Vantagens

- Implementação simples;
- Excelente desempenho em listas pequenas;
- Eficiente para dados quase ordenados;
- Algoritmo estável.

Desvantagens

- Ineficiente para grandes volumes de dados.

Complexidade

- Melhor caso: $O(n)$
- Caso médio: $O(n^2)$
- Pior caso: $O(n^2)$
- **Estabilidade:** Estável.

3.4. Shell Sort

O Shell Sort é uma evolução do Insertion Sort. Seu diferencial consiste na utilização de intervalos denominados gaps, permitindo comparar elementos que estão distantes entre si. Essa estratégia reduz a quantidade de movimentações necessárias e melhora o desempenho da ordenação.

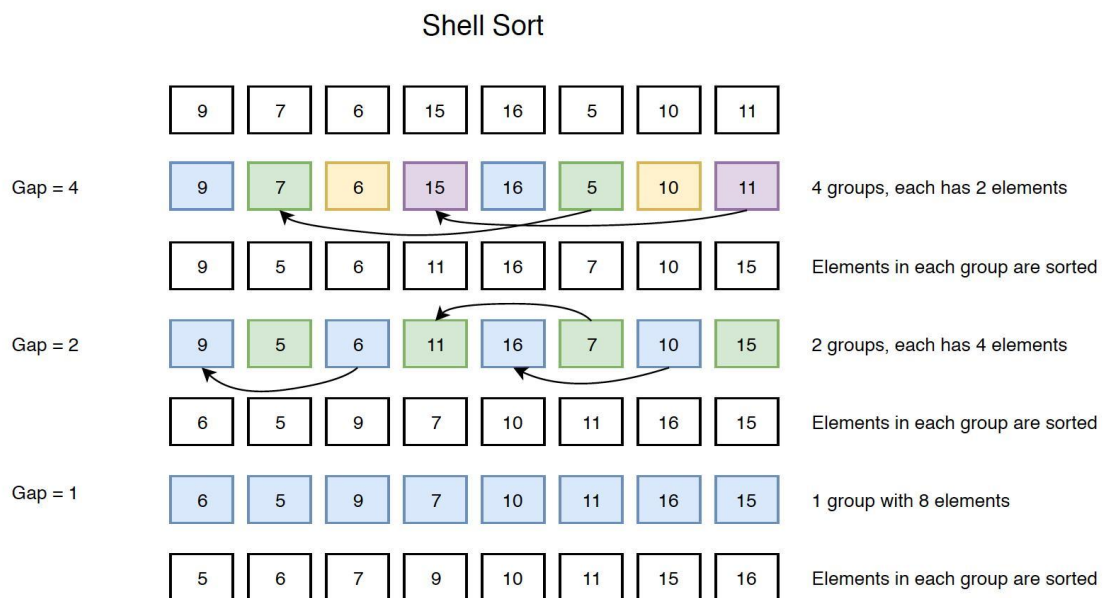


Figura 4 – Funcionamento do Shell Sort

Vantagens

- Mais rápido que os algoritmos elementares;
- Boa eficiência para listas de tamanho médio;
- Relativamente simples de implementar.

Desvantagens

- Seu desempenho depende da sequência de *gaps* utilizada;
- Não possui uma complexidade única definida.

Complexidade

- Melhor caso: aproximadamente $O(n \log n)$
- Caso médio: varia conforme a sequência de gaps
- Pior caso: $O(n^2)$
- **Estabilidade:** Instável.

3.5. Merge Sort

O Merge Sort utiliza a técnica de divisão e conquista (divide and conquer). O vetor é dividido em partes menores, que são ordenadas individualmente e posteriormente combinadas de forma ordenada. Segundo Szwarcfiter e Markenzon (2015), o algoritmo mantém desempenho estável independentemente da disposição inicial dos dados.

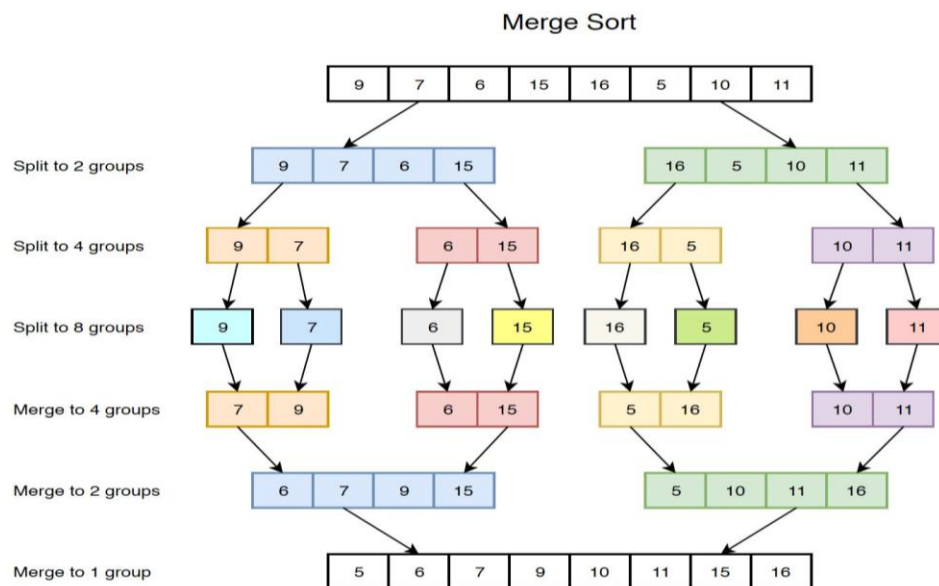


Figura 5 – Funcionamento do Merge Sort

Vantagens

- Excelente desempenho;
- Complexidade previsível;
- Algoritmo estável;
- Adequado para grandes volumes de dados.

Desvantagens

- Necessita memória auxiliar adicional;
- Implementação mais complexa que os algoritmos elementares.

Complexidade

- Melhor caso: $O(n \log n)$
- Caso médio: $O(n \log n)$
- Pior caso: $O(n \log n)$

Estabilidade: Estável.

3.6. Quick Sort

O Quick Sort é um algoritmo de ordenação baseado na técnica de divisão e conquista. Seu funcionamento consiste na escolha de um elemento denominado pivô, a partir do qual os demais elementos são reorganizados em duas partes: uma contendo valores menores e outra contendo valores maiores que o pivô. O processo é repetido recursivamente até que toda a estrutura esteja ordenada (CORMEN et al., 2009).

O funcionamento do Quick Sort pode ser resumido em três etapas: escolha do pivô, particionamento dos elementos e aplicação recursiva do algoritmo às subdivisões geradas.

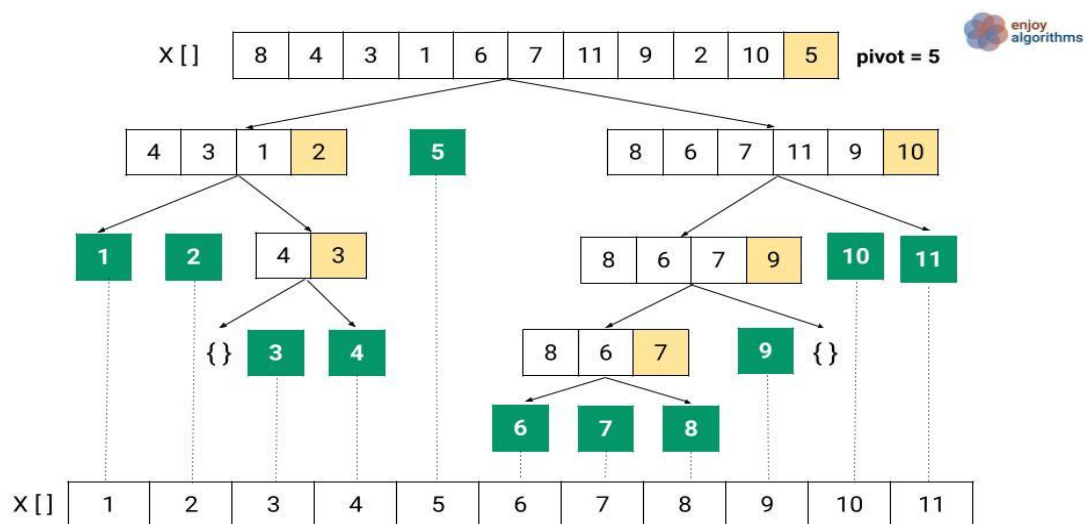


Figura 6 – Funcionamento do Quick Sort

Vantagens

- Muito rápido na prática;
- Excelente desempenho médio;
- Baixo consumo adicional de memória.

Desvantagens

- Pode apresentar desempenho $O(n^2)$ no pior caso;
- A eficiência depende da escolha do pivô.

Complexidade

- Melhor caso: $O(n \log n)$
- Caso médio: $O(n \log n)$
- Pior caso: $O(n^2)$

Estabilidade: Instável.

4. Metodologia Experimental

A presente pesquisa adotou uma abordagem experimental com o objetivo de comparar o desempenho dos algoritmos de ordenação Bubble Sort, Selection Sort, Insertion Sort, Shell Sort, Merge Sort e Quick Sort. Os testes foram realizados por meio de implementações desenvolvidas na linguagem de programação Python, permitindo avaliar a eficiência de cada algoritmo em diferentes cenários.

4.1. Ambiente de Testes

Os algoritmos foram implementados utilizando a linguagem de programação Python devido à sua simplicidade, legibilidade e ampla utilização no ensino e desenvolvimento de algoritmos.

4.2. Métricas de Avaliação

A comparação entre os algoritmos foi realizada com base nas seguintes métricas:

- Tempo de execução;
- Número de comparações entre elementos;
- Número de movimentações (trocas);
- Consumo de memória.

Segundo Cormen et al. (2009), essas métricas são fundamentais para a análise da eficiência de algoritmos de ordenação, pois permitem avaliar tanto o custo computacional quanto o desempenho prático de cada método.

4.3. Implementação dos Algoritmos

Foram desenvolvidas implementações independentes para cada algoritmo de ordenação estudado: Bubble Sort, Selection Sort, Insertion Sort, Shell Sort, Merge Sort e Quick Sort.

Cada algoritmo recebeu como entrada os mesmos conjuntos de dados, garantindo igualdade de condições durante os testes. A medição do tempo de execução foi realizada utilizando funções da biblioteca padrão do Python, permitindo registrar o tempo gasto por cada algoritmo para concluir a ordenação.

5. Aplicações Práticas dos Algoritmos de Ordenação

Os algoritmos de ordenação estão presentes em diversas áreas da tecnologia moderna, desempenhando um papel fundamental na organização e processamento eficiente de informações. A sua utilização permite reduzir o tempo de pesquisa, melhorar o desempenho dos sistemas e facilitar a tomada de decisões em diferentes contextos.

5.1. Aplicações em Motores de Busca

Motores de busca como o Google utilizam algoritmos de ordenação para classificar e apresentar os resultados mais relevantes aos utilizadores. Esses algoritmos analisam fatores como relevância, popularidade, qualidade do conteúdo e histórico de pesquisa, permitindo que a informação mais útil seja apresentada primeiro. Sem técnicas eficientes de ordenação, a obtenção de resultados seria muito mais lenta.

5.2. Aplicações em Sistemas Bancários

As instituições financeiras processam diariamente milhões de transações. Para garantir eficiência e segurança, os algoritmos de ordenação organizam extratos, movimentações financeiras e relatórios contabilísticos. A sua utilização permite localizar informações rapidamente e reduzir o tempo de processamento das operações bancárias.

5.3. Aplicações no Comércio Eletrónico

Nas plataformas de comércio eletrónico, os produtos são organizados segundo critérios como preço, popularidade, avaliações dos clientes e data de lançamento. Essa ordenação facilita a pesquisa dos utilizadores e melhora a experiência de compra, aumentando também a eficiência das vendas online.

5.4. Aplicações em Redes Sociais

As redes sociais utilizam algoritmos de ordenação para organizar publicações, vídeos e comentários apresentados aos utilizadores. A seleção dos conteúdos considera fatores como relevância, número de interações, data de publicação e preferências pessoais, permitindo uma experiência mais personalizada e dinâmica.

CONCLUSÕES

A realização deste trabalho permitiu compreender melhor a importância dos algoritmos e das estruturas de ordenação. Durante o desenvolvimento do tema, foi possível verificar que a organização dos dados é um fator essencial para garantir maior eficiência no processamento das informações e no funcionamento dos sistemas informáticos.

O estudo dos diferentes algoritmos mostrou que cada um possui características próprias e pode ser mais adequado para determinadas situações. Alguns apresentam uma implementação mais simples, enquanto outros oferecem melhor desempenho quando trabalham com grandes quantidades de dados.

Além disso, percebeu-se que os algoritmos de ordenação estão presentes em várias aplicações utilizadas diariamente, como sistemas bancários, bases de dados, motores de busca, redes sociais e sistemas hospitalares. Isso demonstra a sua importância não apenas no contexto acadêmico, mas também no mundo profissional e tecnológico.

Por fim, este trabalho contribuiu para ampliar os conhecimentos sobre a organização de dados e permitiu compreender melhor como os algoritmos de ordenação ajudam a tornar os sistemas mais rápidos, eficientes e capazes de responder às necessidades dos utilizadores.

REFERÊNCIAS

- CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Introduction to Algorithms. 3. ed. Cambridge: MIT Press, 2009.
- SOUZA, J. E. G.; RICARTE, J. V. G.; LIMA, N. C. A. Algoritmos de Ordenação: Um Estudo Comparativo. Anais do Encontro de Computação do Oeste Potiguar – ECOP/UFERSA, 2017.
- SZWARCFITER, J. L.; MARKENZON, L. Estruturas de Dados e Seus Algoritmos. 3. ed. Rio de Janeiro: LTC, 2015.
- ZIVIANI, N. Projeto de Algoritmos com Implementações em Java e C++. São Paulo: Cengage Learning.
- LOUREIRO, A. A. F. Ordenação Interna e Externa. Departamento de Ciência da Computação – UFMG.
- LEVADA, A. Algoritmos e Estruturas de Dados. Universidade Federal de São Carlos – UFSCar.
- freeCodeCamp. Algoritmos de ordenação explicados com exemplos em Python, Java e C++. Disponível em: <https://www.freecodecamp.org>. Acesso em: junho de 2026.
- Hashemi, E. (2024). Sorting algorithms—QuickSort explained. Medium. <https://medium.com/@el.elhamhashemi/sorting-algorithms-quicksort-explained-b5bb814df35b>